Sistemas Operativos

Ayudantía 1: Syscalls y Procesos

Profesor: Víctor Reyes
Ayudante: Diego Banda



Contacto

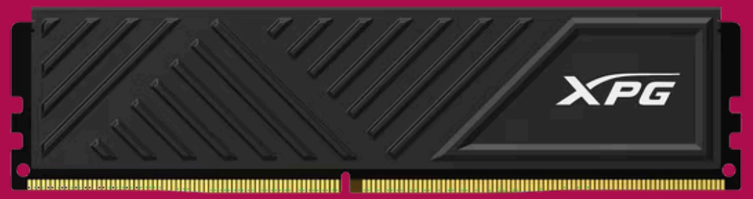
diego.banda@mail.udp.cl

darklouds

Click aquí!!



PC Componentes...









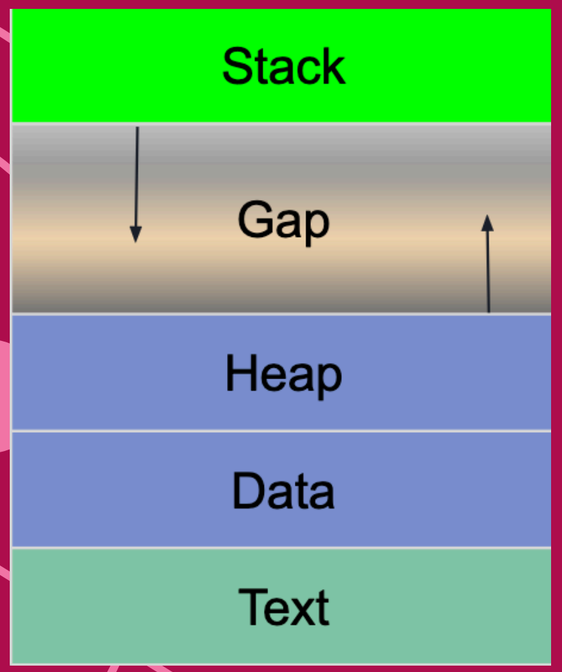


Buses del sistema

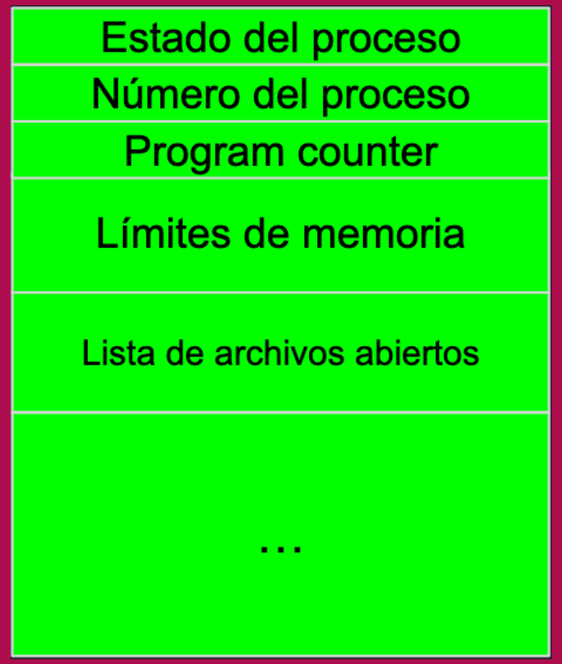
Procesos

Absolutamente todo lo que se ejecuta en tu computador es un proceso, tienen la capacidad de leer, escribir datos, y además contienen datos importantes para su propia ejecución.

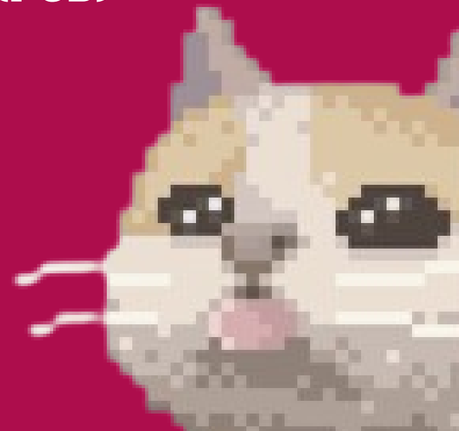
¿Para qué se necesitan?



Proceso en memoria

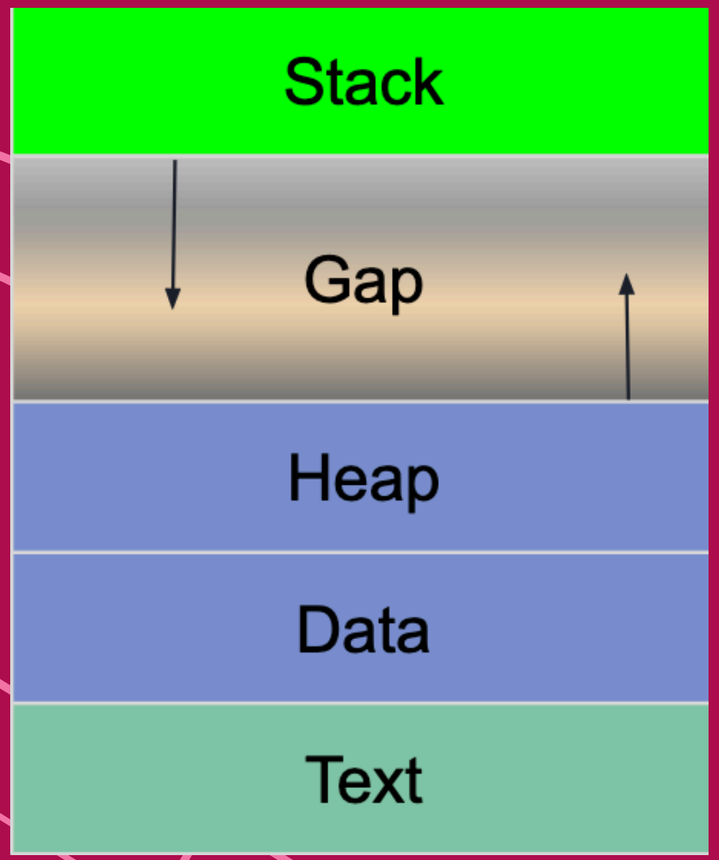


Process Control Block (PCB)



Procesos: Proceso en Memoria

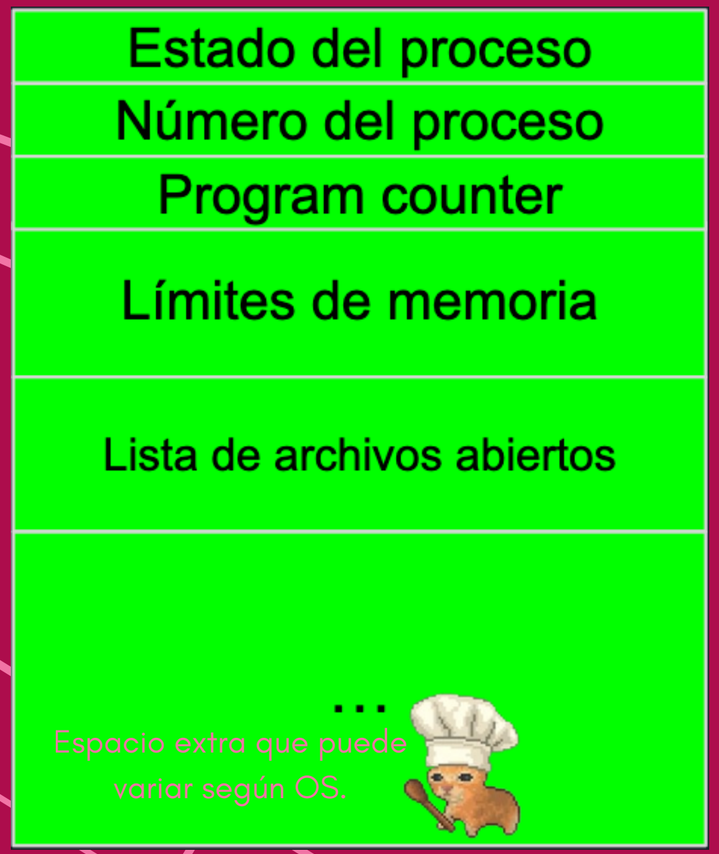
Guarda en memoria RAM lo referente al proceso y lo que necesita este mismo para ejecutarse:



- **Stack:** Guarda variables locales y ejecución de funciones (el stack de ejecución).
- **GAP:** Espacio de memoria libre para permitirle al Stack o al Heap crecer libremente.
- **Heap:** Memoria dinámica creada solicitada durante ejecución (Malloc, new, calloc).
- **Data:** Toda variable estática o creada previamente en el código.
- **Text:** El código a ejecutar, todas las instrucciones que debe seguir el programa.

Procesos: Process Control Block (PCB)

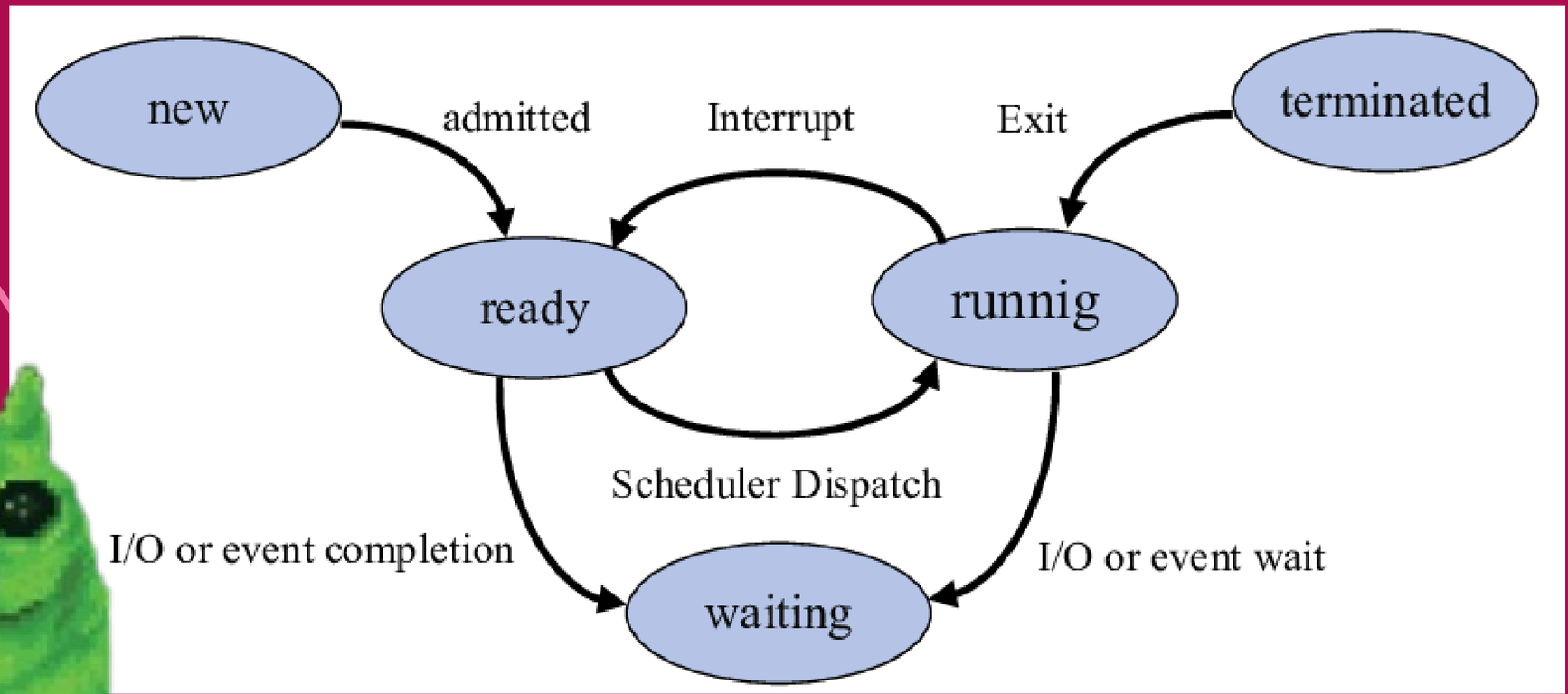
Guarda en memoria RAM lo referente al proceso y lo que necesita este mismo para ejecutarse:



- **Estado del proceso:** Estado actual del proceso (running, waiting, ready, etc).
- **Número del proceso:** Identificador único para cada proceso **PID**. Recuerden: **ÚNICO**.
- **Program Counter:** Siguiente instrucción a ejecutar.
- **Límites de memoria:** Espacio de memoria utilizado por el proceso, este no puede acceder a espacios que no le pertenece,
- **Lista de archivos abiertos:** Qué archivos (o sockets o pipes o dispositivos) está utilizando el archivo.

Procesos: Estados

Todo proceso durante su ejecución pasa por distintos estados según lo requiera y/o decida el OS.

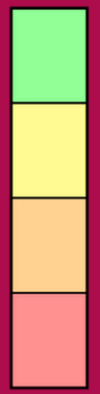




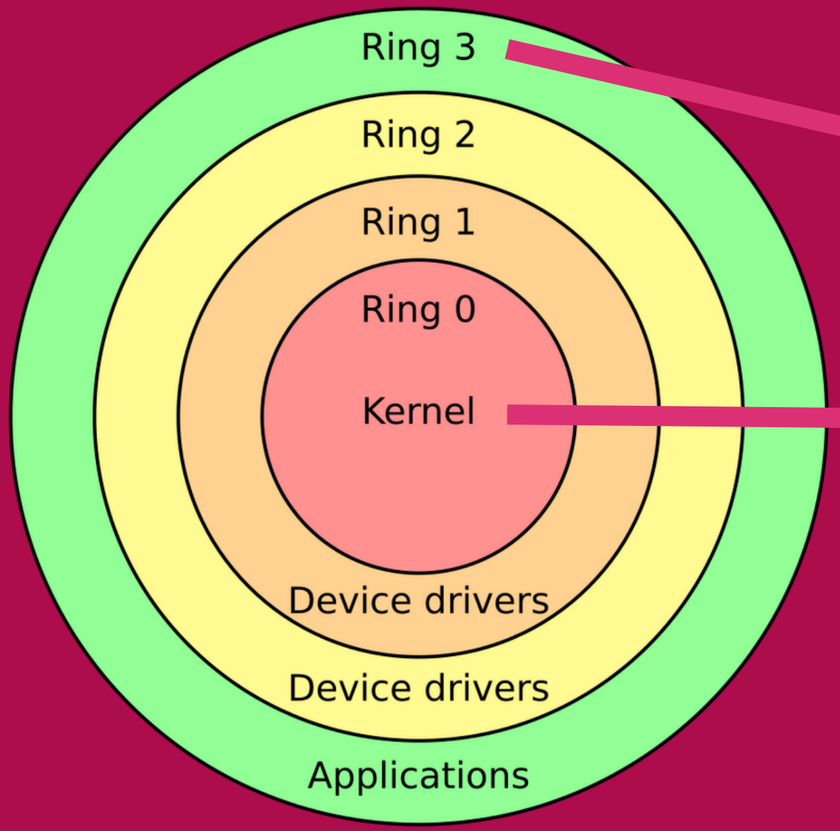
Syscalls

¿Por qué se necesitan de Syscalls? ¿Cuál es la diferencia entre el modo usuario y el modo kernel?

Menos privilegiado
(mayores restricciones)



Más privilegiado
(más acciones o permisos)



El ayudante

El profe

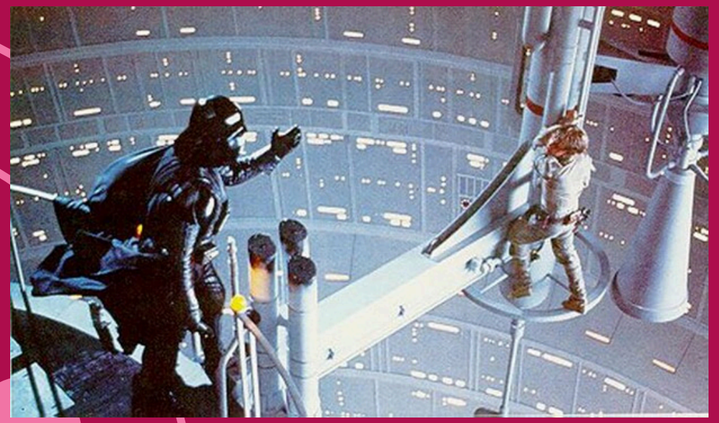
Syscalls

Algunas de las syscalls más importantes y más utilizadas son:

- **fork():** Bifuración, crea una copia del proceso, con exactamente las mismas funciones, variables, etc. Pasan a ser proceso padre (antiguo) y proceso hijo (nuevo).
- **wait():** Un proceso padre espera la finalización de uno de sus procesos hijos.
- **exit():** Termina la ejecución del programa actual.
- **sleep():** Suspende la ejecución de un programa durante un tiempo determinado.
- **getpid():** Retorna el process ID (PID) del actual proceso.

Syscalls: fork()

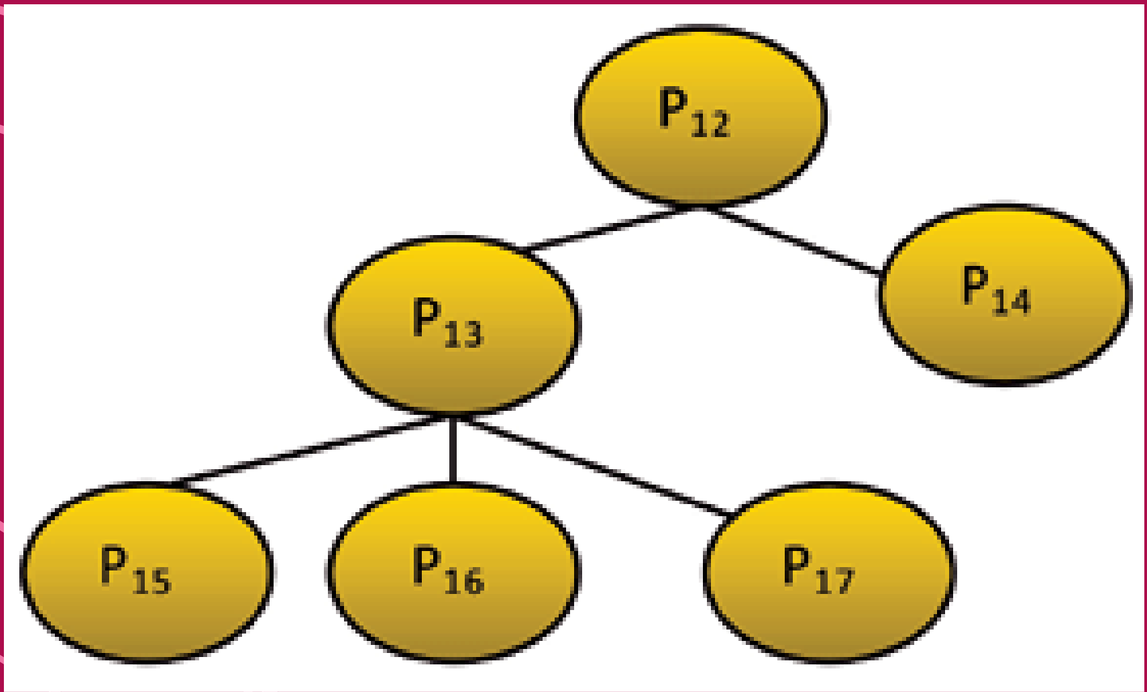
Como se dijo anteriormente, crea un proceso hijo que es una copia idéntica del momento actual en el que se ejecuta, esta función retorna cierto número según lo ocurrido:



- **pid > 0:** Estamos en el proceso padre y el pid es el process id del proceso hijo.
- **pid = 0:** Estamos en el proceso hijo.
- **pid < 0:** Estamos en el proceso padre y hubo un error al crear el proceso hijo.

Procesos: Organización

Todo proceso dentro de los OS están organizados en forma de árbol, por lo tanto, estos siempre tienen que tener un proceso padre al que estar asociados y quizá procesos hijos.



Existen dos casos especiales:

- **Procesos Zombie:** Un proceso pasa a ser zombie cuando el proceso padre aún no lee su salida, por lo tanto está esperando para poder terminar su ejecución.
- **Proceso Huérfano:** Proceso padre termina antes que el proceso hijo, por lo tanto se queda sin padre (y se debe reasignar).

Ejercicios

1.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(){
    pid_t pid;

    for(int i = 0 ; i < 5 ; i++){
        pid = fork();
        if(pid == 0){
            break;
        }else if(pid < 0){
            break;
        }
    }
}
```



Ejercicios

2.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(){
    pid_t pid;

    for(int i = 0 ; i < 5 ; i++){
        pid = fork();
        if(pid > 0){
            break;
        }else if(pid < 0){
            break;
        }
    }
}
```



Ejercicios

3.

Suponga que ejecutamos el siguiente código en C:

```c
int iter = 0;

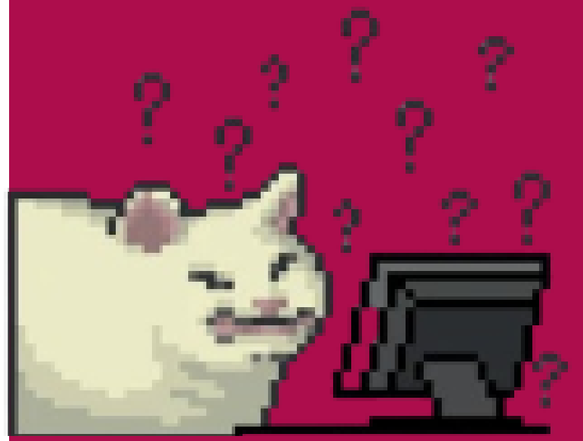
int main(){
    while(iter<2){
        iter++;
        if(fork()){
            printf("Hollow Knight\n");
            if (iter>1){
                pid_t tt = fork();
                if (tt>0){
                    printf("Ender Lilies\n");
                }
                printf("Ender Magnolia\n");
                exit(1);
            }
        }
    }
    printf("Silksong\n");
}
```

(a) **(15 puntos)** ¿Cuántos procesos se crean durante la ejecución del código anterior? Explique (puede ser con palabras o con el árbol de procesos que es creado)

(b) **(15 puntos)** Indique cuántas veces se imprime cada uno de los *strings* del código.

(c) **(15 puntos)** Explique brevemente (3 líneas como máximo) la razón de las distintas salidas, en términos de orden.





Ejercicios

4.

Dado los siguientes códigos en C, en donde se ejecuta el programa *programa1*:

```c
int main() {   //programa1
    pid_t t = fork();
    if (t > 0){
        sleep(6);
        exit(0);
    }
    if (t == 0){
        execlp("./programa2","",NULL);
        fork();
    }
    fork();
}
```

```c
int main() {   //programa2
    printf("Wooooooooohohohoho\n");
    exit(1);
    execlp("./programa1","",NULL);
    fork();
}
```

(15 puntos) Indique la cantidad de procesos que son creados. Explique.



Ejercicios

5.

Asumiendo que existe una asignación justa de la CPU durante la ejecución, ejecutamos el siguiente código en C:

```c

int main(){
    pid_t tt = fork();
    printf("The Knight\n");
    if !(fork() || fork()){
        if (tt > 0){
            tt = fork();
            printf("Pure Vessel\n");
            sleep(5);
            printf("Nightmare King Grimm\n");
        }
        else{
            printf("Hornet\n");
            exit(1);
            printf("The Radiance\n");
        }
    }
    sleep(5);
    printf("You MUST play Hollow Knight and Silksong\n");
}
```

1) ¿Cuántos procesos se crean durante la ejecuición del código anterior?

2) Indique las salidas del código.



Ejercicios

6.

Asumiendo que existe una asignación justa de la CPU durante la ejecución, ejecutamos *programa1*:

```c
int tt = 0;
int main() {  //programa1
    tt = tt + 1;
    pid_t t = fork();
    tt = tt + 1;
    printf("%d\n",tt);
    if (t > 0){
        tt = tt * 2;
        execlp("./programa2","",NULL);
        fork();
        printf("%d\n",tt);
    }
    if (t == 0){
        tt = tt * 3;
        sleep(100);
        printf("%d\n",tt);
    }
    fork();
    printf("%d",t);
}
```

```c
int tt = 0;
int main() {  //programa2
    printf("%d\n",tt);
    exit(1);
    tt = tt + 5;
    execlp("./programa1","",NULL);
    printf("%d\n",tt);
}
```

(a) **(15 puntos)** Muestre el árbol de procesos y qué imprime cada uno de ellos.

(b) **(15 puntos)** ¿Podrían generarse procesos *zombies* y/o *huérfanos* en este escenario? En caso afirmativo, indique en qué momento, cuántos, y justifique su respuesta argumentando en base al código.

